

### About this document

This is the production-ready v2.2 schema. It incorporates all v2.0 and v2.1 corrections plus twelve targeted fixes from the logical consistency review:

- `chk_collected_meta` constraint — refunded+voided\_at gap closed
- `trg_set_collected_at` — extended to cover INSERT as well as UPDATE
- notifications dedup indexes — split to handle NULL job\_id correctly
- `is_low_rated` generated column — NULL propagation made explicit
- `job_cancellation_requests` — UNIQUE relaxed; pending-only partial index added
- `scheduling_conflicts` — canonical ordering constraint prevents mirrored duplicates
- E.2 atomic pair — payment inserted as 'collected' directly, not 'pending'
- `system_config` description tag corrected (key added v2.1, not table)
- `notifications.read_at` — `chk_read_meta` constraint added
- Section F — two missing improvement rows added
- `analytics_processed_events` FK changed to ON DELETE RESTRICT
- `system_config.updated_by` FK changed to ON DELETE SET NULL

PostgreSQL 14+ | NestJS Modular Monolith | Version 2.2

## A. Entity Breakdown

The schema is organised into 25 tables across six logical groups. All entities carry explicit referential integrity, ON DELETE policies, and constraints. Tables added in v2.0 are marked (v2); the table added in v2.1 is marked (v2.1).

| Table                                  | Purpose                                                                                  |
|----------------------------------------|------------------------------------------------------------------------------------------|
| <code>virtual_customers</code>         | Customer identity anchor. Holds the VCID (UUID PK). No PII stored here.                  |
| <code>customer_phones</code>           | All phone numbers linked to a VCID. Many phones per VCID.                                |
| <code>users</code>                     | Owner, Office Staff, Technician. Single table with role enum.                            |
| <code>dealers</code>                   | Separate entity. Has own business details, region, and brand links.                      |
| <code>dealer_credentials</code>        | Auth credentials for dealers, separated from the business entity. (v2)                   |
| <code>dealer_brands</code>             | M:N join between dealers and brands.                                                     |
| <code>brands</code>                    | AC brands. Activatable/deactivatable. Colour-coded.                                      |
| <code>jobs</code>                      | Central entity. Covers both installations and complaints. type + status drive lifecycle. |
| <code>job_units</code>                 | Child rows for multi-unit jobs. One row per AC unit.                                     |
| <code>job_assignments</code>           | Assignment history per job. One active row; superseded rows retained.                    |
| <code>job_cancellation_requests</code> | Dealer-submitted cancellation requests extracted from jobs row. (v2)                     |

| Table                      | Purpose                                                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------------------|
| revisits                   | Child revisit records linked to a parent complaint job.                                                 |
| revisit_assignments        | Per-revisit technician assignment history.                                                              |
| payments                   | 1:1 with job. Records amount, method, status, collected timestamp, and edit trail.                      |
| payment_methods            | Owner-managed list of accepted payment methods.                                                         |
| job_timeline               | Append-only audit log for every significant job event.                                                  |
| notifications              | In-app notification store with deduplication.                                                           |
| scheduling_conflicts       | Log of acknowledged scheduling conflicts. (v2 — DDL was missing in v1)                                  |
| analytics_technician_daily | Precomputed daily rollup per technician.                                                                |
| analytics_business_daily   | Precomputed daily rollup for business-wide metrics.                                                     |
| analytics_brand_daily      | Precomputed daily rollup per brand.                                                                     |
| analytics_dealer_daily     | Precomputed daily rollup per dealer.                                                                    |
| analytics_processed_events | Idempotency ledger for the analytics worker. 90-day rolling retention. (v2.1)                           |
| system_config              | Key-value store for owner-managed operational thresholds. standard_job_duration_mins key added in v2.1. |
| customer_reviews           | Optional Module 19. Post-completion review links and ratings.                                           |

## B. Full Database Schema (DDL)

All DDL is standard PostgreSQL 14+. Run sections in order shown. Enum types must be created before the tables that reference them.

### B.1 — Enums

#### v2.1 change — payment\_status

Extended from 2 values (pending, collected) to 4 values (pending, collected, refunded, disputed).

Adding these values now avoids a disruptive ALTER TYPE under production load.

For existing v2.0 databases, run these two statements as standalone (non-transactional) migrations:

```
ALTER TYPE payment_status ADD VALUE 'refunded' AFTER 'collected';
```

```
ALTER TYPE payment_status ADD VALUE 'disputed' AFTER 'refunded';
```

```
-- =====
-- ENUMS
-- =====

CREATE TYPE user_role AS ENUM ('owner','office_staff','technician');

CREATE TYPE job_type AS ENUM ('installation','complaint');

CREATE TYPE job_source AS ENUM ('direct','via_dealer');
```

```

-- Combined job status enum (v1 stored this as TEXT – now a proper enum).
-- The validate_job_status trigger enforces per-type validity.
CREATE TYPE job_status AS ENUM (
    'pending_schedule', 'scheduled', 'assigned', 'acknowledged',
    'in_transit', 'in_process', 'completed',
    'new', 'resolved', 'needs_revisit', 'revisit_scheduled',
    'resolved_on_revisit', 'cancellation_requested', 'cancelled'
);

-- — payment_status (v2.1: extended with refunded and disputed) —
--
-- State transition rules (enforced by service layer):
--   pending   → collected   technician completes job
--   pending   → disputed    payment contested before collection
--   collected → refunded    owner authorises refund
--   collected → disputed    payment contested after collection
--   disputed  → collected   dispute resolved in favour of business
--   disputed  → refunded    dispute resolved in favour of customer
CREATE TYPE payment_status AS ENUM (
    'pending',
    'collected',
    'refunded',
    'disputed'
);

-- Renamed from visit_outcome to avoid collision with column names.
CREATE TYPE arrival_outcome AS ENUM ('on_time', 'late', 'no_show', 'rescheduled');

CREATE TYPE revisit_reason AS ENUM (
    'part_unavailable', 'customer_not_home', 'issue_recurring',
    'further_diagnosis_required', 'custom'
);

-- Replaced TEXT CHECK in v1.
CREATE TYPE revisit_outcome AS ENUM ('resolved', 'needs_revisit');

CREATE TYPE cancellation_request_status AS ENUM
('pending', 'approved', 'rejected');

CREATE TYPE timeline_event_type AS ENUM (
    'status_transition', 'status_undo', 'status_rollback',
    'assignment', 'reassignment', 'scheduling_conflict_ack',
    'payment_recorded', 'payment_edited', 'payment_void',
    'payment_retained', 'cancellation', 'cancellation_request',
    'cancellation_approved', 'cancellation_rejected', 'reopened',
    'customer_linked', 'customer_link_dismissed', 'note_added',
    'revisit_created', 'revisit_scheduled', 'rollback_payment_voided',
    'system_event' -- background-worker entries with no human actor
);

CREATE TYPE notification_event AS ENUM (
    'dealer_job_submitted', 'job_assigned',
    'cancellation_request_submitted', 'cancellation_request_outcome',
    'job_unacknowledged', 'no_show_flagged',
    'repeat_complaint_detected', 'frequent_complaint_detected',
    'third_revisit_reached'
);

CREATE TYPE review_mode AS ENUM ('off', 'optional', 'mandatory');

```

## B.2 — Customer Identity

```
CREATE TABLE virtual_customers (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  is_frequent BOOLEAN      NOT NULL DEFAULT FALSE,  
  frequent_flagged_at TIMESTAMPTZ,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE customer_phones (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  vcid        UUID          NOT NULL  
    REFERENCES virtual_customers(id) ON DELETE RESTRICT,  
  phone       TEXT          NOT NULL,  
  is_primary  BOOLEAN      NOT NULL DEFAULT FALSE,  
  linked_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  -- FK enforced (was a bare comment in v1)  
  linked_by   UUID          REFERENCES users(id) ON DELETE SET NULL,  
  UNIQUE (vcid, phone)  
);  
  
CREATE INDEX idx_customer_phones_phone ON customer_phones (phone);  
CREATE INDEX idx_customer_phones_vcid  ON customer_phones (vcid);
```

## B.3 — Users, Dealers & Brands

```
CREATE TABLE users (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  role        user_role    NOT NULL,  
  full_name   TEXT          NOT NULL,  
  email       TEXT          UNIQUE NOT NULL,  
  password_hash TEXT        NOT NULL,  
  is_active   BOOLEAN      NOT NULL DEFAULT TRUE,  
  is_deleted  BOOLEAN      NOT NULL DEFAULT FALSE,  
  created_by  UUID          REFERENCES users(id) ON DELETE SET NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
-- Auth credentials extracted to dealer_credentials.  
-- Allows dealers without portal access and future OAuth/SSO  
-- without altering the dealers table.  
CREATE TABLE dealers (  
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),  
  business_name TEXT        NOT NULL,  
  contact_name TEXT        NOT NULL,  
  email       TEXT          UNIQUE NOT NULL,  
  region      TEXT,  
  is_active   BOOLEAN      NOT NULL DEFAULT TRUE,  
  is_deleted  BOOLEAN      NOT NULL DEFAULT FALSE,  
  created_by  UUID          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW())
```

```
);

CREATE TABLE dealer_credentials (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id    UUID          NOT NULL UNIQUE
                    REFERENCES dealers(id) ON DELETE CASCADE,
    password_hash TEXT        NOT NULL,
    last_login_at TIMESTAMPTZ,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE brands (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    name        TEXT          NOT NULL UNIQUE,
    colour_hex  CHAR(6),
    is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
    is_deleted  BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE dealer_brands (
    dealer_id UUID NOT NULL REFERENCES dealers(id) ON DELETE CASCADE,
    brand_id  UUID NOT NULL REFERENCES brands(id)  ON DELETE RESTRICT,
    PRIMARY KEY (dealer_id, brand_id)
);
```

## B.4 — Jobs (Core Entity)

```
CREATE TABLE jobs (
    id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    type        job_type     NOT NULL,
    source      job_source   NOT NULL,
    -- Proper enum (was TEXT in v1).
    -- validate_job_status trigger enforces per-type validity.
    status      job_status   NOT NULL,
    brand_id    UUID          NOT NULL REFERENCES brands(id) ON DELETE RESTRICT,
    dealer_id   UUID          REFERENCES dealers(id) ON DELETE RESTRICT,
    vcid        UUID          NOT NULL REFERENCES virtual_customers(id) ON DELETE
    RESTRICT,

    -- Embedded PII (see Assumption A-1 in Section G)
    customer_name TEXT        NOT NULL,
    address      TEXT         NOT NULL,

    -- Flags derived from VCID history.
    -- Context columns record the threshold in effect when each flag was set
    -- so that retroactive config changes can be audited.
    is_repeat    BOOLEAN NOT NULL DEFAULT FALSE,
    repeat_window_days_used INT,
    is_frequent  BOOLEAN NOT NULL DEFAULT FALSE,
    frequent_threshold_used INT,
    frequent_window_used INT,
    -- Auto-set by trg_chronic when revisit sequence_number >= 3
    is_chronic   BOOLEAN NOT NULL DEFAULT FALSE,
```

```

scheduled_at      TIMESTAMPTZ,
actual_arrival    TIMESTAMPTZ,
visit_outcome     arrival_outcome,
late_by_minutes   INT CHECK (late_by_minutes >= 0),
issue_description TEXT,

-- Direct cancellation (dealer requests go to job_cancellation_requests)
cancelled_at      TIMESTAMPTZ,
cancelled_by      UUID REFERENCES users(id) ON DELETE SET NULL,
cancellation_reason TEXT,

version          INT          NOT NULL DEFAULT 0,
created_by       UUID          REFERENCES users(id) ON DELETE SET NULL,
created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
is_deleted       BOOLEAN       NOT NULL DEFAULT FALSE,

CONSTRAINT chk_dealer_source
    CHECK (source <> 'via_dealer' OR dealer_id IS NOT NULL),
CONSTRAINT chk_cancel_meta
    CHECK (cancelled_at IS NULL OR
           (cancelled_by IS NOT NULL AND cancellation_reason IS NOT NULL)),
CONSTRAINT chk_cancel_order
    CHECK (cancelled_at IS NULL OR cancelled_at >= created_at)
);

-- All 6 indexes from v1 Section D (were listed but not created in v1 DDL)
CREATE INDEX idx_jobs_vcid_created ON jobs (vcid, created_at DESC) WHERE
is_deleted = FALSE;
CREATE INDEX idx_jobs_status_type ON jobs (status, type) WHERE
is_deleted = FALSE;
CREATE INDEX idx_jobs_dealer ON jobs (dealer_id, created_at DESC) WHERE
is_deleted = FALSE;
CREATE INDEX idx_jobs_brand ON jobs (brand_id);
CREATE INDEX idx_jobs_scheduled ON jobs (scheduled_at);
CREATE INDEX idx_jobs_is_chronic ON jobs (is_chronic) WHERE is_chronic =
TRUE;
CREATE INDEX idx_jobs_not_deleted ON jobs (id) WHERE is_deleted = FALSE;

```

## B.4a — Job Cancellation Requests

```

CREATE TABLE job_cancellation_requests (
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID          NOT NULL
                        REFERENCES jobs(id) ON DELETE RESTRICT,
    -- NOTE: UNIQUE(job_id) removed. A dealer may submit a new request after
    -- rejection. The partial index below enforces at most one PENDING request
    -- per job at a time – consistent with the revisits pattern.
    requested_by_dealer_id UUID      NOT NULL
                        REFERENCES dealers(id) ON DELETE RESTRICT,
    reason            TEXT          NOT NULL,
    requested_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    status            cancellation_request_status NOT NULL DEFAULT 'pending',
    decided_by        UUID          REFERENCES users(id) ON DELETE SET NULL,
    decided_at        TIMESTAMPTZ,
    decision_note      TEXT,
    CONSTRAINT chk_decision_meta
        CHECK (
            (status = 'pending' AND decided_by IS NULL AND decided_at IS NULL) OR

```

```

        (status <> 'pending' AND decided_by IS NOT NULL AND decided_at IS NOT
NULL)
    ),
    CONSTRAINT chk_decided_order
        CHECK (decided_at IS NULL OR decided_at >= requested_at)
);

-- Enforces at most one pending cancellation request per job at any time.
-- Does not prevent a second request after the first is approved/rejected.
CREATE UNIQUE INDEX idx_cancel_req_one_pending
    ON job_cancellation_requests (job_id)
    WHERE status = 'pending';

CREATE INDEX idx_cancel_req_job ON job_cancellation_requests (job_id);

```

## B.4b — Job Units

```

CREATE TABLE job_units (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID                NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
    model             TEXT                NOT NULL,
    unit_type         TEXT                NOT NULL,
    num_units         SMALLINT CHECK (num_units > 0),
    fault_description TEXT,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_job_units_job ON job_units (job_id);

```

## B.5 — Assignments

```

CREATE TABLE job_assignments (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID                NOT NULL REFERENCES jobs(id) ON DELETE RESTRICT,
    technician_id     UUID                NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    is_active         BOOLEAN             NOT NULL DEFAULT TRUE,
    assigned_by       UUID                NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    assigned_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    acknowledged_at   TIMESTAMPTZ,
    unassigned_at     TIMESTAMPTZ
);

-- Enforces exactly one active assignment per job.
CREATE UNIQUE INDEX idx_job_assignments_active
    ON job_assignments (job_id) WHERE is_active = TRUE;
CREATE INDEX idx_job_assignments_technician ON job_assignments (technician_id);
CREATE INDEX idx_job_assignments_job        ON job_assignments (job_id);

```

## B.6 — Revisits

```

CREATE TABLE revisits (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id            UUID                NOT NULL REFERENCES jobs(id) ON DELETE
RESTRICT,
    sequence_number   SMALLINT            NOT NULL,
    reason            revisit_reason     NOT NULL,

```

```

    custom_reason    TEXT,
    scheduled_at     TIMESTAMPTZ,
    actual_arrival   TIMESTAMPTZ,
    visit_outcome    arrival_outcome,
    late_by_minutes  INT CHECK (late_by_minutes >= 0),
    outcome          revisit_outcome,
    notes            TEXT,
    created_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at       TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_custom_reason
        CHECK (reason <> 'custom' OR custom_reason IS NOT NULL),
    UNIQUE (job_id, sequence_number)
);
CREATE INDEX idx_revisits_job ON revisits (job_id);

-- — Chronic flag trigger —————
-- Fires AFTER INSERT on revisits.
-- When a job accumulates 3 or more revisits it is classified as
-- chronic and jobs.is_chronic is set to TRUE.
--
-- Out-of-order insert safety (offline sync):
--   If sequence 3 arrives before sequences 1 or 2 due to sync lag,
--   the trigger fires correctly on sequence 3. When the earlier
--   sequences subsequently arrive they have sequence_number < 3
--   and the IF condition is FALSE – they are silently skipped.
--   The AND is_chronic = FALSE guard prevents any double-update.
--   No schema change required. Confirmed sufficient per design review.

CREATE OR REPLACE FUNCTION fn_check_chronic()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.sequence_number >= 3 THEN
        UPDATE jobs
        SET     is_chronic = TRUE,
              updated_at = NOW()
        WHERE  id          = NEW.job_id
              AND is_chronic = FALSE; -- no-op if already flagged
    END IF;
    RETURN NEW;
END;
$$;

-- AFTER INSERT (not BEFORE): sequence_number is committed to
-- revisits first; a failure in the UPDATE rolls back both.
CREATE TRIGGER trg_chronic
AFTER INSERT ON revisits
FOR EACH ROW EXECUTE FUNCTION fn_check_chronic();

CREATE TABLE revisit_assignments (
    id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    revisit_id        UUID          NOT NULL REFERENCES revisits(id) ON DELETE
RESTRICT,
    technician_id     UUID          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    is_active         BOOLEAN       NOT NULL DEFAULT TRUE,
    assigned_by        UUID          NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    assigned_at       TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    acknowledged_at   TIMESTAMPTZ,
    unassigned_at     TIMESTAMPTZ
);

```



```
CREATE UNIQUE INDEX idx_revisit_assignments_active
ON revisit_assignments (revisit_id) WHERE is_active = TRUE;
```

## B.7 — Payments

### v2.1 changes — payments table

1. collected\_at TIMESTAMPTZ column added.
2. payment\_status enum now includes refunded and disputed.
3. chk\_collected\_meta constraint relaxed: collected\_at is PRESERVED on void, refund, and dispute — never cleared — so auditors retain the full lifecycle paper trail: recorded\_at → collected\_at → voided\_at.
4. trg\_set\_collected\_at trigger added: auto-populates collected\_at on pending → collected; does NOT clear it on any subsequent transition.

```
CREATE TABLE payment_methods (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT          NOT NULL UNIQUE,
  is_active   BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE TABLE payments (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL UNIQUE
                      REFERENCES jobs(id) ON DELETE RESTRICT,
  amount      NUMERIC(12,2) NOT NULL CHECK (amount >= 0),
  payment_method_id UUID    NOT NULL
                      REFERENCES payment_methods(id) ON DELETE RESTRICT,
  status      payment_status NOT NULL DEFAULT 'pending',
  recorded_by  UUID          NOT NULL
                      REFERENCES users(id) ON DELETE RESTRICT,
  recorded_at  TIMESTAMPTZ   NOT NULL DEFAULT NOW(),

  -- Populated automatically by trg_set_collected_at when status
  -- first transitions to 'collected'. Never cleared after that point.
  -- Preserved through void, refund, and dispute for audit purposes.
  collected_at TIMESTAMPTZ,

  -- Tracks who last edited the payment (office staff corrections).
  last_edited_by  UUID          REFERENCES users(id) ON DELETE SET NULL,
  last_edited_at  TIMESTAMPTZ,

  -- Void fields
  voided_at       TIMESTAMPTZ,
  voided_by       UUID          REFERENCES users(id) ON DELETE SET NULL,
  void_reason     TEXT,

  -- Retain fields
  retained_note   TEXT,

  -- Optimistic locking
  version         INT          NOT NULL DEFAULT 0,
  updated_at      TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
```

```

-- — Constraints —————

CONSTRAINT chk_void_meta
CHECK (voided_at IS NULL OR
      (voided_by IS NOT NULL AND void_reason IS NOT NULL)),

CONSTRAINT chk_edit_meta
CHECK (last_edited_at IS NULL OR last_edited_by IS NOT NULL),

-- collected_at rules:
--   REQUIRED when status = 'collected' (money received)
--   REQUIRED when status = 'refunded' (must have been collected first)
--       NOTE: refunded branch is stated explicitly and takes priority
--       over the voided_at catch-all, preventing a refunded+NULL gap.
--   OPTIONAL when status = 'disputed' (may arise from pending or collected)
--   PRESERVED when status = 'pending' AND voided_at IS NOT NULL (reversed
before collection)
--   ABSENT when status = 'pending' AND voided_at IS NULL
CONSTRAINT chk_collected_meta
CHECK (
  -- Refunded ALWAYS requires collected_at, even if also voided
  (status = 'refunded' AND collected_at IS NOT NULL)
  OR
  -- Collected requires collected_at
  (status = 'collected' AND collected_at IS NOT NULL)
  OR
  -- Disputed: collected_at optional (may arise pre- or post-collection)
  (status = 'disputed')
  OR
  -- Pending + voided: collected_at preserved if previously set, absent if
not
  (status = 'pending' AND voided_at IS NOT NULL)
  OR
  -- Pending + no void: collected_at must be absent
  (status = 'pending' AND voided_at IS NULL AND collected_at IS NULL)
)
);

CREATE INDEX idx_payments_job ON payments (job_id);

-- — Trigger: auto-populate collected_at —————
-- BEFORE INSERT OR UPDATE so it covers all entry points.
-- Handling both INSERT and UPDATE prevents gaps from data migrations,
-- test fixtures, or future code paths that insert directly as 'collected'.
-- COALESCE guard makes it idempotent on re-fire.
-- Does NOT clear collected_at on reversal – once money has been
-- touched that timestamp is permanent (audit requirement).

CREATE OR REPLACE FUNCTION fn_set_collected_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  -- Entering 'collected' for the first time (INSERT or UPDATE).
  -- TG_OP check handles INSERT where OLD is NULL.
  IF NEW.status = 'collected'
    AND (TG_OP = 'INSERT' OR OLD.status <> 'collected')
  THEN
    NEW.collected_at := COALESCE(NEW.collected_at, NOW());
  END IF;
  -- All other transitions: collected_at is left exactly as-is.

```

```

-- Covers: collected→voided, collected→refunded,
--         collected→disputed, pending→disputed.
RETURN NEW;
END;
$$;

CREATE TRIGGER trg_set_collected_at
BEFORE INSERT OR UPDATE OF status ON payments
FOR EACH ROW EXECUTE FUNCTION fn_set_collected_at();

```

## B.8 — Timeline (Audit Log)

### Rescheduling auditability

job\_timeline already satisfies the requirement for storing old\_value, new\_value, and change\_type for schedule changes. No structural change required.

old\_value → previous\_value JSONB e.g. {"scheduled\_at": "2024-06-01T09:00:00Z"}

new\_value → new\_value JSONB e.g. {"scheduled\_at": "2024-06-02T14:00:00Z"}

change\_type → event\_type enum value: status\_transition (or rescheduled if added)

The service layer must write a job\_timeline row with both JSONB columns populated on every scheduled\_at update.

```

-- Append-only. Rows are never updated or deleted.
-- actor_user_id or actor_dealer_id must be set,
-- OR event_type must be 'system_event' (background worker).

CREATE TABLE job_timeline (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id            UUID                NOT NULL
    REFERENCES jobs(id) ON DELETE RESTRICT,
  event_type        timeline_event_type NOT NULL,
  actor_user_id     UUID                REFERENCES users(id)   ON DELETE SET NULL,
  actor_dealer_id   UUID                REFERENCES dealers(id) ON DELETE SET NULL,
  previous_value     JSONB,
  new_value          JSONB,
  reason            TEXT,
  related_job_id     UUID                REFERENCES jobs(id)   ON DELETE SET NULL,
  revisit_id         UUID                REFERENCES revisits(id) ON DELETE SET NULL,
  note_text          TEXT,
  occurred_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_actor
    CHECK (
      actor_user_id IS NOT NULL OR
      actor_dealer_id IS NOT NULL OR
      event_type = 'system_event'
    )
);

CREATE INDEX idx_timeline_job            ON job_timeline (job_id, occurred_at DESC);
CREATE INDEX idx_timeline_event_type    ON job_timeline (event_type);
CREATE INDEX idx_timeline_actor_user    ON job_timeline (actor_user_id);

```

## B.9 — Notifications

```

CREATE TABLE notifications (
  id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
  event_type        notification_event NOT NULL,
  job_id            UUID REFERENCES jobs(id)    ON DELETE CASCADE,
  recipient_user_id UUID REFERENCES users(id)   ON DELETE CASCADE,
  recipient_dealer_id UUID REFERENCES dealers(id) ON DELETE CASCADE,
  is_read           BOOLEAN             NOT NULL DEFAULT FALSE,
  read_at          TIMESTAMPTZ,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_recipient
    CHECK (recipient_user_id IS NOT NULL OR recipient_dealer_id IS NOT NULL),
  -- read_at must be populated whenever is_read = TRUE.
  -- Prevents is_read = TRUE rows with no legible timestamp.
  CONSTRAINT chk_read_meta
    CHECK (is_read = FALSE OR read_at IS NOT NULL)
);

-- — Deduplication indexes —————
-- Split into four partial indexes (job-scoped vs. non-job-scoped) because
-- PostgreSQL treats NULLs as distinct in unique indexes — two rows with
-- the same event_type and recipient but job_id IS NULL would never conflict
-- in a single index, defeating deduplication for system-level notifications.

-- User recipient, job-scoped (e.g. job_assigned)
CREATE UNIQUE INDEX idx_notif_dedup_user_job
  ON notifications (event_type, job_id, recipient_user_id)
  WHERE recipient_user_id IS NOT NULL AND job_id IS NOT NULL;

-- User recipient, non-job-scoped (e.g. frequent_complaint_detected)
CREATE UNIQUE INDEX idx_notif_dedup_user_nojob
  ON notifications (event_type, recipient_user_id)
  WHERE recipient_user_id IS NOT NULL AND job_id IS NULL;

-- Dealer recipient, job-scoped
CREATE UNIQUE INDEX idx_notif_dedup_dealer_job
  ON notifications (event_type, job_id, recipient_dealer_id)
  WHERE recipient_dealer_id IS NOT NULL AND job_id IS NOT NULL;

-- Dealer recipient, non-job-scoped
CREATE UNIQUE INDEX idx_notif_dedup_dealer_nojob
  ON notifications (event_type, recipient_dealer_id)
  WHERE recipient_dealer_id IS NOT NULL AND job_id IS NULL;

CREATE INDEX idx_notifications_recipient_user
  ON notifications (recipient_user_id, is_read, created_at DESC);
CREATE INDEX idx_notifications_recipient_dealer
  ON notifications (recipient_dealer_id, is_read, created_at DESC);
CREATE INDEX idx_notifications_job ON notifications (job_id);

```

## B.10 — Scheduling Conflicts

```

-- Present in v1 entity list but DDL was missing. Defined in v2.
-- v2.2: canonical ordering constraint added — job_id must always be the
-- lesser UUID so (A,B) and (B,A) are stored identically, preventing
-- double-counting in analytics and duplicate conflict records.
-- Service layer must insert as: (LEAST(jobA,jobB), GREATEST(jobA,jobB)).
CREATE TABLE scheduling_conflicts (

```

```

    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    job_id             UUID                NOT NULL REFERENCES jobs(id) ON DELETE
RESTRIC,
    conflicting_job_id UUID                NOT NULL REFERENCES jobs(id) ON DELETE
RESTRIC,
    technician_id      UUID                NOT NULL REFERENCES users(id) ON DELETE
RESTRIC,
    acknowledged_by    UUID                NOT NULL REFERENCES users(id) ON DELETE
RESTRIC,
    acknowledged_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    note               TEXT,
    CONSTRAINT chk_not_self_conflict
        CHECK (job_id <> conflicting_job_id),
    -- Enforces canonical ordering: job_id < conflicting_job_id (UUID
lexicographic).
    -- Combined with UNIQUE, this guarantees a conflict pair (A,B) == (B,A).
    CONSTRAINT chk_canonical_order
        CHECK (job_id < conflicting_job_id),
    UNIQUE (job_id, conflicting_job_id, technician_id)
);
CREATE INDEX idx_conflicts_job ON scheduling_conflicts (job_id);
-- Also index conflicting_job_id for reverse lookups:
CREATE INDEX idx_conflicts_conflicting ON scheduling_conflicts
(conflicting_job_id);

```

## B.11 — Analytics Processed Events

### v2.1 — new table

Promoted from an inline code comment in v2.0 Section E.3 to a first-class DDL section.

Includes a 90-day rolling retention function and pg\_cron scheduling.

Rationale: the analytics worker is unlikely ever to retry an event older than a few days. 90 days provides a generous safety margin for catastrophic recovery.

Without retention, this table grows at 1 row per timeline event — the fastest-growing table in the schema — and will degrade index performance over time.

```

-- -----
-- ANALYTICS PROCESSED EVENTS
-- -----
-- Idempotency ledger for the analytics worker.
-- One row per job_timeline event applied to the analytics_*_daily
-- rollup tables. On retry the worker checks this table first and
-- skips already-processed events, preventing double-counting.
--
-- Owned exclusively by the analytics worker.
-- No application business logic reads or writes this table.

CREATE TABLE analytics_processed_events (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    timeline_event_id UUID                NOT NULL UNIQUE
        REFERENCES job_timeline(id) ON DELETE RESTRICT,
    -- ON DELETE RESTRICT (not CASCADE): job_timeline rows are immutable.
    -- If a timeline row were ever deleted (emergency surgery), the analytics
    -- team must manually audit the rollup tables before removing this record.
    -- Silent CASCADE would allow the worker to re-process the event and
    -- double-count analytics — defeating the idempotency guarantee.

```

```

    processed_at          TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Primary lookup: 'has this event been processed?' O(log n)
CREATE UNIQUE INDEX idx_ape_timeline_event
    ON analytics_processed_events (timeline_event_id);

-- Secondary: monitoring and backfill queries
CREATE INDEX idx_ape_processed_at
    ON analytics_processed_events (processed_at DESC);

-- — 90-day rolling retention —————

CREATE OR REPLACE FUNCTION fn_purge_analytics_processed_events()
RETURNS INT LANGUAGE plpgsql AS $$
DECLARE deleted_count INT;
BEGIN
    DELETE FROM analytics_processed_events
    WHERE processed_at < NOW() - INTERVAL '90 days';
    GET DIAGNOSTICS deleted_count = ROW_COUNT;
    RAISE NOTICE 'analytics_processed_events purge: % rows deleted',
deleted_count;
    RETURN deleted_count;
END;
$$;

-- — pg_cron scheduling (run once during deployment setup) —————
-- Requires: CREATE EXTENSION IF NOT EXISTS pg_cron;
-- Runs daily at 02:00 UTC (low-traffic window).
--
-- SELECT cron.schedule(
--     'purge-analytics-processed-events',
--     '0 2 * * *',
--     'SELECT fn_purge_analytics_processed_events()'
-- );
--
-- Verify: SELECT * FROM cron.job
--         WHERE jobname = 'purge-analytics-processed-events';
--
-- Alternative (if pg_cron unavailable):
-- Call fn_purge_analytics_processed_events() from the NestJS analytics
-- worker via @nestjs/schedule @Cron('0 2 * * *').

```

## B.12 — Analytics Rollup Tables

```

-- Dashboard queries hit these tables, not jobs directly.
-- Updated by NestJS workers after every job mutation.
-- See Section E for idempotent UPSERT pattern.

CREATE TABLE analytics_technician_daily (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    technician_id     UUID                NOT NULL
                                REFERENCES users(id) ON DELETE RESTRICT,
    date              DATE                NOT NULL,
    jobs_assigned     INT                  NOT NULL DEFAULT 0,
    installations_completed INT            NOT NULL DEFAULT 0,
    complaints_resolved INT                NOT NULL DEFAULT 0,

```

```

    revisit_count            INT            NOT NULL DEFAULT 0,
    no_show_count            INT            NOT NULL DEFAULT 0,
    on_time_count            INT            NOT NULL DEFAULT 0,
    late_count               INT            NOT NULL DEFAULT 0,
    total_payment_collected NUMERIC(14,2) NOT NULL DEFAULT 0,
    pending_payment_total    NUMERIC(14,2) NOT NULL DEFAULT 0,
    avg_resolution_mins      NUMERIC(10,2),
    avg_acknowledgment_mins NUMERIC(10,2),
    first_visit_resolved     INT            NOT NULL DEFAULT 0,
    avg_star_rating          NUMERIC(3,2), -- Module 19; NULL when feature off
    updated_at               TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    UNIQUE (technician_id, date)
);

-- Trigger: reject non-technician rows.
CREATE OR REPLACE FUNCTION fn_validate_technician_role()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM users WHERE id = NEW.technician_id AND role = 'technician'
    ) THEN
        RAISE EXCEPTION
            'analytics_technician_daily: technician_id % is not a technician',
            NEW.technician_id;
    END IF;
    RETURN NEW;
END;
$$;
CREATE TRIGGER trg_validate_technician_role
BEFORE INSERT OR UPDATE ON analytics_technician_daily
FOR EACH ROW EXECUTE FUNCTION fn_validate_technician_role();

CREATE TABLE analytics_business_daily (
    id                UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    date              DATE            NOT NULL UNIQUE,
    total_jobs        INT            NOT NULL DEFAULT 0,
    installations_completed INT        NOT NULL DEFAULT 0,
    complaints_resolved INT          NOT NULL DEFAULT 0,
    revenue           NUMERIC(14,2) NOT NULL DEFAULT 0,
    pending_payment_total NUMERIC(14,2) NOT NULL DEFAULT 0,
    direct_jobs       INT            NOT NULL DEFAULT 0,
    dealer_jobs       INT            NOT NULL DEFAULT 0,
    repeat_complaint_count INT        NOT NULL DEFAULT 0,
    frequent_customer_count INT        NOT NULL DEFAULT 0,
    avg_resolution_mins NUMERIC(10,2),
    on_time_rate       NUMERIC(5,4),
    updated_at        TIMESTAMPTZ    NOT NULL DEFAULT NOW()
);

CREATE TABLE analytics_brand_daily (
    id                UUID            PRIMARY KEY DEFAULT gen_random_uuid(),
    brand_id          UUID            NOT NULL REFERENCES brands(id) ON DELETE
RESTRICT,
    date              DATE            NOT NULL,
    installations_completed INT        NOT NULL DEFAULT 0,
    complaints_received INT          NOT NULL DEFAULT 0,
    complaints_resolved INT          NOT NULL DEFAULT 0,
    revenue           NUMERIC(14,2) NOT NULL DEFAULT 0,
    revisit_count     INT            NOT NULL DEFAULT 0,
    updated_at        TIMESTAMPTZ    NOT NULL DEFAULT NOW(),

```

```

    UNIQUE (brand_id, date)
);

CREATE TABLE analytics_dealer_daily (
    id                UUID                PRIMARY KEY DEFAULT gen_random_uuid(),
    dealer_id         UUID                NOT NULL REFERENCES dealers(id) ON DELETE
RESTRICT,
    date              DATE                NOT NULL,
    jobs_submitted    INT                 NOT NULL DEFAULT 0,
    complaints_submitted INT             NOT NULL DEFAULT 0,
    installations_submitted INT           NOT NULL DEFAULT 0,
    resolved_count     INT                 NOT NULL DEFAULT 0,
    avg_resolution_mins NUMERIC(10,2),
    pending_count      INT                 NOT NULL DEFAULT 0,
    updated_at         TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    UNIQUE (dealer_id, date)
);

CREATE INDEX idx_tech_daily_date    ON analytics_technician_daily (date);
CREATE INDEX idx_biz_daily_date    ON analytics_business_daily   (date);
CREATE INDEX idx_brand_daily_date  ON analytics_brand_daily      (brand_id,
date);
CREATE INDEX idx_dealer_daily_date ON analytics_dealer_daily     (dealer_id,
date);

```

## B.13 — System Config

### v2.1 change — standard\_job\_duration\_mins

New config key added to seed function (default 120 minutes).

Used by the scheduling engine to detect technician overlap conflicts and compute SLA breach risk. Service-layer fallback resolution order:

1. Brand override → brand\_configurations.job\_duration\_mins (extension point, not yet introduced)
2. Job-type override → future job\_type\_configurations table
3. This global default

Cast at service layer: parseInt(value, 10) with COALESCE fallback to 120.

```

CREATE TABLE system_config (
    key          TEXT          PRIMARY KEY,
    value        TEXT          NOT NULL,
    -- ON DELETE SET NULL: allows owner account replacement without blocking
    -- a migration. A NULL updated_by means 'original seed / user no longer
    -- exists'; updated_at still records when the value was last changed.
    updated_by   UUID          REFERENCES users(id) ON DELETE SET NULL,
    updated_at   TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

-- seed_system_config: safe to re-run (ON CONFLICT DO NOTHING).
-- Call after the owner user row is inserted:
-- SELECT seed_system_config('<actual-owner-uuid>');
CREATE OR REPLACE FUNCTION seed_system_config(p_owner_id UUID)
RETURNS VOID LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO system_config (key, value, updated_by) VALUES
        ('repeat_complaint_window_days', '30', p_owner_id),

```



```

('frequent_complaint_threshold', '3', p_owner_id),
('frequent_complaint_window_days', '90', p_owner_id),
('punctuality_grace_period_mins', '15', p_owner_id),
('customer_review_mode', 'off', p_owner_id),
('undo_window_seconds', '60', p_owner_id),
-- Global default for expected job visit duration in minutes.
-- Used by the scheduling engine for overlap detection and
-- SLA breach risk windows. Integer stored as TEXT per
-- system_config convention; cast at service layer.
('standard_job_duration_mins', '120', p_owner_id)
ON CONFLICT (key) DO NOTHING;
END;
$$;

-- — Extension point: brand-specific duration overrides —————
-- When brand-specific durations become a hard requirement, introduce:
--
-- CREATE TABLE brand_configurations (
--   brand_id          UUID          PRIMARY KEY REFERENCES brands(id) ON DELETE
-- CASCADE,
--   job_duration_mins INT          NOT NULL CHECK (job_duration_mins > 0),
--   updated_by        UUID          NOT NULL REFERENCES users(id),
--   updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
-- );
--
-- Service layer checks brand_configurations first, then falls back
-- to system_config.standard_job_duration_mins.

```

## B.14 — Customer Reviews (Module 19)

```

CREATE TABLE customer_reviews (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      UUID          NOT NULL UNIQUE REFERENCES jobs(id) ON DELETE
  RESTRICT,
  technician_id UUID          NOT NULL REFERENCES users(id) ON DELETE
  RESTRICT,
  brand_id    UUID          NOT NULL REFERENCES brands(id) ON DELETE
  RESTRICT,
  review_token UUID          NOT NULL UNIQUE DEFAULT gen_random_uuid(),
  link_generated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- Generated column enforces the 48-hour window.
  expires_at    TIMESTAMPTZ
  GENERATED ALWAYS AS (link_generated_at + INTERVAL '48 hours') STORED,
  submitted_at  TIMESTAMPTZ,
  star_rating   SMALLINT CHECK (star_rating BETWEEN 1 AND 5),
  -- NULL star_rating (review not yet submitted) produces NULL is_low_rated,
  -- not FALSE. CASE makes this explicit so future maintainers cannot
  -- accidentally treat NULL as 'not low-rated' in dashboard queries.
  is_low_rated  BOOLEAN
  GENERATED ALWAYS AS (
    CASE WHEN star_rating IS NOT NULL THEN star_rating <= 2
    ELSE NULL -- NULL = not yet rated; distinct from FALSE = rated, not
low
    END
  ) STORED,
  review_text   TEXT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()

```

```
);
CREATE INDEX idx_reviews_technician ON customer_reviews (technician_id);
CREATE INDEX idx_reviews_lowRated
  ON customer_reviews (is_lowRated) WHERE is_lowRated = TRUE;
```

## B.15 — State & Lifecycle Enforcement

```
-- jobs.status is now a proper enum (job_status), so only known
-- values are accepted at the column level.
-- The trigger below adds per-type validity as a second layer.

CREATE OR REPLACE FUNCTION validate_job_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
  valid_installation job_status[] := ARRAY[
    'pending_schedule', 'scheduled', 'assigned', 'acknowledged',
    'in_transit', 'in_process', 'completed', 'cancelled'
  ]::job_status[];
  valid_complaint job_status[] := ARRAY[
    'new', 'assigned', 'acknowledged', 'in_transit', 'in_process',
    'resolved', 'needs_revisit', 'revisit_scheduled', 'resolved_on_revisit',
    'cancellation_requested', 'cancelled'
  ]::job_status[];
BEGIN
  IF NEW.type = 'installation' AND NOT (NEW.status = ANY(valid_installation))
  THEN
    RAISE EXCEPTION 'Invalid installation status: %', NEW.status;
  END IF;
  IF NEW.type = 'complaint' AND NOT (NEW.status = ANY(valid_complaint)) THEN
    RAISE EXCEPTION 'Invalid complaint status: %', NEW.status;
  END IF;
  RETURN NEW;
END;
$$;
CREATE TRIGGER trg_validate_job_status
  BEFORE INSERT OR UPDATE OF status ON jobs
  FOR EACH ROW EXECUTE FUNCTION validate_job_status();
```

## C. Relationships

| From              | To                        | Cardinality      | ON DELETE policy                                 |
|-------------------|---------------------------|------------------|--------------------------------------------------|
| virtual_customers | customer_phones           | 1 : N            | RESTRICT — cannot remove VCID while phones exist |
| virtual_customers | jobs                      | 1 : N            | RESTRICT — cannot remove VCID while jobs exist   |
| jobs              | job_units                 | 1 : N            | CASCADE — units owned by job                     |
| jobs              | job_assignments           | 1 : N (1 active) | RESTRICT — preserve assignment history           |
| jobs              | job_cancellation_requests | 1 : 0..1         | RESTRICT — UNIQUE enforces at most one request   |

| From            | To                         | Cardinality      | ON DELETE policy                                                                    |
|-----------------|----------------------------|------------------|-------------------------------------------------------------------------------------|
| jobs            | revisits                   | 1 : N            | RESTRICT — preserve revisit history                                                 |
| revisits        | revisit_assignments        | 1 : N (1 active) | RESTRICT                                                                            |
| jobs            | payments                   | 1 : 1            | RESTRICT — UNIQUE on payments.job_id                                                |
| jobs            | job_timeline               | 1 : N            | RESTRICT — audit rows are immutable                                                 |
| jobs            | notifications              | 1 : N            | CASCADE — notifications follow their job                                            |
| jobs            | scheduling_conflicts       | 1 : N            | RESTRICT                                                                            |
| jobs            | customer_reviews           | 1 : 0..1         | RESTRICT — Module 19                                                                |
| users           | job_assignments            | 1 : N            | Technician assigned to many jobs                                                    |
| dealers         | dealer_credentials         | 1 : 1            | CASCADE — credentials owned by dealer                                               |
| dealers         | jobs                       | 1 : N            | RESTRICT                                                                            |
| dealers         | dealer_brands              | N : M            | via dealer_brands join table                                                        |
| brands          | jobs                       | 1 : N            | RESTRICT                                                                            |
| payment_methods | payments                   | 1 : N            | RESTRICT                                                                            |
| job_timeline    | analytics_processed_events | 1 : 0..1         | RESTRICT — deletion of a timeline row is blocked if a processed-event record exists |

## D. Indexing Strategy

All indexes described in v1 Section D now exist in the DDL. Partial indexes on `is_active = TRUE` and `is_deleted = FALSE` keep index sizes small and exclude soft-deleted / superseded rows from operational queries.

| Index target                               | Type              | Justification                                                 |
|--------------------------------------------|-------------------|---------------------------------------------------------------|
| customer_phones.phone                      | B-tree            | Phone-number lookup for VCID resolution on every new job      |
| jobs(vcid, created_at DESC) WHERE !deleted | Partial composite | Repeat/frequent detection — most latency-sensitive query path |
| jobs(status, type) WHERE !deleted          | Partial composite | Dashboard job-list filtering                                  |
| jobs(dealer_id, created_at DESC)           | Composite         | Dealer history view                                           |
| jobs(brand_id)                             | B-tree            | Brand analytics and filtering                                 |
| jobs(scheduled_at)                         | B-tree            | Scheduling conflict window checks                             |
| jobs(is_chronic) WHERE TRUE                | Partial           | Chronic-job dashboard queries                                 |

| Index target                                                                    | Type           | Justification                                                                        |
|---------------------------------------------------------------------------------|----------------|--------------------------------------------------------------------------------------|
| jobs(id) WHERE !deleted                                                         | Partial        | Excludes soft-deleted rows from all operational queries                              |
| job_assignments(job_id) WHERE active                                            | Partial unique | Enforces single active assignment; O(1) current-technician lookup                    |
| job_timeline(job_id, occurred_at DESC)                                          | Composite      | Ordered retrieval of all events per job                                              |
| notifications dedup — 4 partial indexes (event_type, job_id/nojob, recipient_*) | Partial unique | Split into job-scoped and non-job-scoped pairs; handles NULL job_id correctly (v2.2) |
| notifications(recipient_*, is_read, created_at DESC)                            | Composite      | Badge count and unread list                                                          |
| analytics_*_daily(date)                                                         | B-tree         | Date-range dashboard queries on all analytics tables                                 |
| customer_reviews.is_lowRated WHERE TRUE                                         | Partial        | Owner dashboard low-rating filter                                                    |
| analytics_processed_events(timeline_event_id)                                   | Unique         | Idempotency check — O(log n) existence lookup (v2.1)                                 |
| analytics_processed_events(processed_at DESC)                                   | B-tree         | Monitoring, backfill, and retention purge queries (v2.1)                             |

## E. Concurrency & Data Integrity

### E.1 — Optimistic Locking

Both jobs and payments use a version counter. The service layer reads the current version, passes it back on update, and raises HTTP 409 if the version has changed. The payment UPDATE guard was absent in v1 and is now enforced.

```
-- Jobs (unchanged from v1)
UPDATE jobs
SET    status      = :new_status,
       updated_at  = NOW(),
       version     = version + 1
WHERE  id          = :job_id
       AND version = :expected_version
       AND is_deleted = FALSE;
-- rowcount = 0 → raise OptimisticLockException → HTTP 409

-- Payments (version guard was missing in v1 — now enforced)
UPDATE payments
SET    amount      = :amount,
       payment_method_id = :method_id,
       last_edited_by   = :editor_id,
       last_edited_at   = NOW(),
       updated_at      = NOW(),
       version        = version + 1
WHERE  job_id       = :job_id
       AND version   = :expected_version;
-- rowcount = 0 → raise OptimisticLockException → HTTP 409
```

## E.2 — Payment + Status Atomic Pair

The payment is inserted directly as 'collected' within the same transaction as the job status update. This collapses the previous two-round-trip pattern (insert as pending → update to collected) into a single atomic operation, closing the gap where a job could be permanently in completed+pending-payment if the second UPDATE was lost.

```
BEGIN;
-- 1. Advance job status with version guard
UPDATE jobs
SET     status = 'completed', version = version + 1, updated_at = NOW()
WHERE   id = :job_id AND version = :v AND is_deleted = FALSE;
-- If rowcount = 0 → ROLLBACK immediately (optimistic lock conflict)

-- 2. Insert payment directly as 'collected'.
-- trg_set_collected_at fires on INSERT (covers TG_OP = 'INSERT'),
-- populating collected_at = NOW() automatically.
-- UNIQUE on job_id prevents double-payment on retry.
INSERT INTO payments
  (job_id, amount, payment_method_id, status, recorded_by)
VALUES
  (:job_id, :amount, :method_id, 'collected', :technician_id);
-- collected_at is set by trigger; no second UPDATE required.

-- 3. Append timeline entries
INSERT INTO job_timeline (job_id, event_type, actor_user_id, new_value)
VALUES (:job_id, 'status_transition', :technician_id,
        '{"status": "completed"}'::jsonb);
INSERT INTO job_timeline (job_id, event_type, actor_user_id, new_value)
VALUES (:job_id, 'payment_recorded', :technician_id,
        json_build_object('amount', :amount, 'method_id', :method_id)::jsonb);
COMMIT;
-- All three operations succeed together or all roll back.
-- No intermediate state where job is completed but payment is pending.
```

## E.3 — Analytics Idempotency

The `analytics_processed_events` table (Section B.11) is the idempotency ledger. The worker checks it before each UPSERT and marks events after applying them.

```
-- Worker: fetch unprocessed timeline events
SELECT jt.id, jt.event_type, jt.job_id, jt.occurred_at
FROM   job_timeline jt
WHERE  jt.event_type IN ('status_transition', 'payment_recorded')
AND    NOT EXISTS (
  SELECT 1 FROM analytics_processed_events ape
  WHERE   ape.timeline_event_id = jt.id
)
ORDER BY jt.occurred_at
LIMIT  500;

-- After UPSERT into analytics_*_daily tables, mark as processed:
INSERT INTO analytics_processed_events (timeline_event_id, processed_at)
VALUES (:timeline_id, NOW())
```

```

ON CONFLICT (timeline_event_id) DO NOTHING;

-- Example: update technician daily when a complaint is resolved
INSERT INTO analytics_technician_daily
    (technician_id, date, complaints_resolved, total_payment_collected)
VALUES (:technician_id, CURRENT_DATE, 1, :amount)
ON CONFLICT (technician_id, date) DO UPDATE
    SET complaints_resolved      = analytics_technician_daily.complaints_resolved +
1,
    total_payment_collected =
analytics_technician_daily.total_payment_collected + :amount,
    updated_at                = NOW();

```

## F. Improvements Summary

All rows below represent changes from v1.0 through v2.1. v2.0 fixes are listed first, followed by v2.1 patch decisions.

| Severity        | Issue                                                 | Fix applied                                                                         |
|-----------------|-------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>CRITICAL</b> | No ON DELETE behaviour on any FK                      | Explicit ON DELETE<br>RESTRICT/CASCADE/SET NULL on every FK                         |
| <b>CRITICAL</b> | jobs.status stored as TEXT                            | Replaced with job_status enum; trigger operates on typed values                     |
| <b>CRITICAL</b> | customer_phones.linked_by not an FK                   | Added REFERENCES users(id) ON DELETE SET NULL                                       |
| <b>CRITICAL</b> | payments version guard missing                        | UPDATE payments ... WHERE version = :expected_version enforced                      |
| <b>HIGH</b>     | is_repeat/is_frequent stale after threshold change    | Context columns added:<br>repeat_window_days_used,<br>frequent_threshold_used       |
| <b>HIGH</b>     | is_chronic never auto-set                             | trg_chronic AFTER INSERT on revisits sets jobs.is_chronic when sequence_number >= 3 |
| <b>HIGH</b>     | revisits.outcome was TEXT CHECK                       | Replaced with revisit_outcome enum                                                  |
| <b>HIGH</b>     | scheduling_conflicts DDL missing                      | Full CREATE TABLE with FKs, unique constraint, and index added in B.10              |
| <b>HIGH</b>     | All 6 jobs indexes absent from DDL                    | All indexes from v1 Section D now created in B.4                                    |
| <b>HIGH</b>     | payments.recorded_by cannot track edits               | Added last_edited_by / last_edited_at with chk_edit_meta                            |
| <b>HIGH</b>     | customer_reviews.expires_at unconstrained             | GENERATED ALWAYS AS<br>(link_generated_at + INTERVAL '48 hours')<br>STORED          |
| <b>HIGH</b>     | analytics_technician_daily allows non-technician rows | trg_validate_technician_role trigger on INSERT/UPDATE                               |
| <b>MEDIUM</b>   | PII spread / GDPR risk                                | Context columns added; GDPR note documented in Assumptions                          |

| Severity  | Issue                                              | Fix applied                                                                                        |
|-----------|----------------------------------------------------|----------------------------------------------------------------------------------------------------|
| MEDIUM    | job_timeline both actor columns nullable           | chk_actor constraint: at least one actor or event_type = 'system_event'                            |
| MEDIUM    | Dealer password_hash in business table             | Extracted to dealer_credentials; supports future auth extension                                    |
| MEDIUM    | visit_outcome enum name collides with column       | Enum renamed to arrival_outcome throughout                                                         |
| MEDIUM    | system_config seed uses placeholder UUID           | Replaced with seed_system_config(p_owner_id UUID) function                                         |
| MEDIUM    | avg_star_rating Module 19 leaky abstraction        | Column retained in base schema; behaviour controlled by system_config.customer_review_mode         |
| RISK      | Inconsistent soft-delete semantics                 | is_deleted added to users, dealers, brands for uniform soft-delete                                 |
| RISK      | Cancellation columns bloating jobs row             | Extracted to job_cancellation_requests child table                                                 |
| RISK      | No deduplication on notifications                  | idx_notif_dedup_user/dealer partial unique indexes (split for NULL job_id in v2.2)                 |
| RISK      | Analytics UPSERT double-counts on retry            | analytics_processed_events + idempotent worker pattern in E.3                                      |
| v2.1      | payment_status enum had only 2 values              | Extended with refunded and disputed; state transitions documented                                  |
| v2.1      | payments.collected_at missing                      | Column added; trg_set_collected_at auto-populates on collected transition; never cleared for audit |
| v2.1      | chk_collected_meta too strict (cleared on void)    | Relaxed: collected_at preserved on void, refund, dispute — full lifecycle paper trail              |
| v2.1      | analytics_processed_events was informal            | Promoted to first-class DDL in B.11 with retention function and pg_cron schedule                   |
| v2.1      | standard_job_duration_mins not seeded              | Added to seed_system_config() with default 120; fallback hierarchy documented                      |
| v2.1      | trg_chronic specification not annotated            | Trigger body surfaced with out-of-order safety confirmation in B.6                                 |
| v2.1      | Rescheduling audit coverage unclear                | Confirmed: previous_value + new_value JSONB cover old/new; documented in B.8                       |
| v2.2 CRIT | chk_collected_meta: refunded+voided_at gap         | Refunded branch now stated explicitly before voided_at catch-all; order-of-evaluation gap closed   |
| v2.2 CRIT | trg_set_collected_at only fired on UPDATE          | Extended to BEFORE INSERT OR UPDATE; TG_OP guard handles INSERT where OLD is NULL                  |
| v2.2 HIGH | Notifications dedup indexes broken for NULL job_id | Four partial indexes (user+job, user+nojob, dealer+job, dealer+nojob) replace two                  |
| v2.2 HIGH | is_low_rated: NULL propagation implicit            | CASE expression makes NULL = not yet rated explicit; distinguishes from FALSE = not low-rated      |

| Severity  | Issue                                                                 | Fix applied                                                                                        |
|-----------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| v2.2 HIGH | job_cancellation_requests: UNIQUE blocks re-request after rejection   | UNIQUE(job_id) removed; pending-only partial unique index allows multiple lifecycle requests       |
| v2.2 HIGH | scheduling_conflicts: mirrored duplicate pairs possible               | chk_canonical_order CHECK enforces job_id < conflicting_job_id; index on conflicting_job_id added  |
| v2.2 HIGH | E.2 atomic pair: payment inserted as pending then updated separately  | Payment now inserted directly as collected; trigger fires on INSERT; no two-round-trip gap         |
| v2.2 MED  | system_config row labelled (v2.1) incorrectly                         | Tag corrected to 'key added v2.1'; table itself existed since v1                                   |
| v2.2 MED  | notifications.read_at unconstrained when is_read = TRUE               | chk_read_meta CHECK added: is_read = FALSE OR read_at IS NOT NULL                                  |
| v2.2 MED  | Section F missing avg_star_rating and soft-delete rows                | Two previously omitted improvement rows added                                                      |
| v2.2 RISK | analytics_processed_events FK was ON DELETE CASCADE                   | Changed to ON DELETE RESTRICT; silent cascade would re-enable double-counting on timeline deletion |
| v2.2 RISK | system_config.updated_by ON DELETE RESTRICT blocked owner replacement | Changed to ON DELETE SET NULL; NULL updated_by = original seed user no longer exists               |

## G. Assumptions & Design Decisions

### A-1 — Embedded PII retained by design

customer\_name and address remain embedded in jobs. GDPR/right-to-erasure compliance must be handled at the application layer by nullifying these fields on verified erasure requests. A migration to a separate customer\_profiles table is recommended when the user base exceeds ~50k VCIDs.

### A-2 — Repeat/frequent flags are point-in-time

is\_repeat and is\_frequent are set at job-creation time. Context columns (repeat\_window\_days\_used, frequent\_threshold\_used, frequent\_window\_used) record which threshold was in effect. If retroactive recalculation is required after a threshold change, a background job must be run.

### A-3 — collected\_at is permanent once set

Per design review decision: collected\_at is never cleared after initial population, even on void, refund, or dispute. This preserves the full payment lifecycle paper trail:

- recorded\_at = when the technician logged the payment
- collected\_at = when money was marked as received
- voided\_at = when the transaction was reversed (if applicable)

The chk\_collected\_meta constraint and trg\_set\_collected\_at trigger both enforce this.



#### A-4 — `standard_job_duration_mins` is a global default

Kept in `system_config` as a single global value. Brand-specific overrides are a tracked extension point (`brand_configurations` table stub in B.13). When introduced, no changes to `system_config` or jobs are required — the service layer checks `brand_configurations` first and falls back to the global config value.

#### A-5 — `analytics_processed_events` retention is 90 days

Rows older than 90 days are purged by `fn_purge_analytics_processed_events()`, scheduled via `pg_cron` at 02:00 UTC daily. If `pg_cron` is unavailable, the NestJS analytics worker must invoke the function via `@nestjs/schedule`. Retention period can be adjusted in the function body without a migration.

#### A-6 — `ON DELETE RESTRICT` is the default policy

All FKs use `ON DELETE RESTRICT` unless noted otherwise. Physical deletion of any operational entity (user, dealer, brand, job) is intentionally blocked at the DB level. Deactivation via `is_active = FALSE` / `is_deleted = TRUE` is the correct operational path.

#### A-7 — `payment_status` enum migration note

For existing v2.0 databases, the two new enum values must be added as standalone non-transactional statements (not inside `BEGIN...COMMIT` with other DDL):

```
ALTER TYPE payment_status ADD VALUE 'refunded' AFTER 'collected';  
ALTER TYPE payment_status ADD VALUE 'disputed' AFTER 'refunded';
```

Confirm your migration tool supports non-transactional DDL, or place these in a separate migration file marked as non-transactional.

#### A-8 — `scheduling_conflicts`: service layer must enforce canonical ordering

The `chk_canonical_order` constraint (`job_id < conflicting_job_id`) is enforced at the DB level but the service layer must also be aware. Always insert with:

```
job_id          = LEAST(jobA_uuid, jobB_uuid)  
conflicting_job_id = GREATEST(jobA_uuid, jobB_uuid)
```

UUID lexicographic comparison is deterministic in PostgreSQL. A service-layer unit test should assert this ordering for every scheduling conflict creation path.

#### A-9 — `system_config.updated_by` is now nullable

`updated_by` changed from `NOT NULL REFERENCES ... ON DELETE RESTRICT` to nullable `REFERENCES ... ON DELETE SET NULL`. A NULL value means the seeding user no longer exists (e.g. original owner account was replaced during a migration).

`updated_at` still records when the value was last changed, preserving the audit trail.

Application code reading `system_config` should handle NULL `updated_by` gracefully.